School of Computing
FACULTY OF ENGINEERING AND
PHYSICAL SCIENCES



Final Report

Evaluating Machine Learning Algorithms For Solving The Inverse Kinematics Problem

Abdulla Ali Moledina

Submitted in accordance with the requirements for the degree of BSc (Hons) Computer Science (Industrial)

2024/25

COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (06/05/2025)
Link to online code repository	URL	Sent to supervisor and assessor (06/05/2025)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) Abdulla Ali Moledina

© 2024/25 The University of Leeds and Abdulla Ali Moledina

Summary

Solving the inverse kinematics problem is one of the most difficult problems those designing robots encounter. Finding an efficient solution to the inverse kinematics problem would make designing robots much easier. With the advancements in Machine Learning made over the past decade, we can use these algorithms to potentially find solutions to this problem in a quicker, and much more efficient way. This paper aims to use a robot simulator created in PyBullet to solve the inverse kinematics problem using its built-in function, and compare the answers against those gained from using various machine learning techniques to solve the same problem, and evaluate the results to find out which technique provides the best solution in terms of accuracy and speed.



Acknowledgements

I would like to thank my supervisor, Dr Yanlong Huang, for helping me come up with and shape this project idea, and for his support throughout this project. I am also grateful to my assessor, Massimiliano Fasi, for his advice and support throughout our meetings, helping me to ensure that this report and this project are of the best quality. I would also like to thank Shaokang Wu for being able to create and provide the dataset used in this project, as well as helping me find details about the real robot arm used to produce the dataset.

I would like to thank my family - Bashaer, Travis, Farwa, Asgher, Sukaina, Ahlaam, Aliabbas, Mama and Baba - as well as my two nieces, Sofia and Elif. You have supported and encouraged me all my life, teaching me important lessons that have shaped me into the man I am today. Thank you for everything.

Finally, I would like to dedicate this paper to the following:

My late aunt, Fatima Ali Raza Nathani
My late uncle, Hassan Ali Raza Nathani
My late grandfather, Mohammed G.H. Moledina
My late uncle, Roshan Mohammed G.H. Moledina
My late aunt, Rubab Ali Raza Nathani

Contents

1 Introduction and Background Research **1**

1.1 Introduction . . . 1

1.2 Robotics . . . 1

1.2.1 History of Robotics . . . 1

1.2.2 The Role of Software in Robotics . . . 2

1.3 Kinematics . . . 2

1.3.1 The Inverse Kinematics Problem . . . 3

1.3.2 Importance of Inverse Kinematics . . . 3

1.3.3 Closed-form Solutions of the Inverse Kinematics Problem . . . 3

1.3.4 Numerical Solutions of the Inverse Kinematics Problem . . . 5

1.4 Quaternion Space . . . 6

1.5 Machine Learning . . . 7

1.5.1 Supervised Learning Algorithms . . . 7

1.5.2 Ensemble Methods . . . 8

2 Methodology **9**

2.1 Finding a Dataset . . . 9

2.1.1 Data Processing . . . 9

2.2 Finding a Suitable Robotics Simulator . . . 9

2.2.1 PyBullet . . . 10

2.2.2 Robotic End-Effector Design . . . 10

2.3 Choice of Machine Learning Models . . . 10

2.3.1 k-Nearest Neighbours (kNN) . . . 11

2.3.2 Linear Regression . . . 11

2.3.3 Neural Network . . . 11

2.3.4 Decision Trees . . . 11

2.3.5 Support Vector Regression (SVR) . . . 11

2.3.6 Random Forest . . . 11

2.3.7 Gradient Boosting . . . 12

2.3.8 Gaussian Process Regression (GPR) . . . 12

2.3.9 Bayesian Linear Regression (BLR) . . . 12

2.3.10 Lasso Regression . . . 12

2.3.11 Ridge Regression . . . 12

2.3.12 Kernel Ridge Regression (KRR) . . . 13

2.4 Metrics Used to Evaluate Models . . . 13

2.4.1 Mean Absolute Error (MAE) . . . 13

2.4.2 Mean Squared Error (MSE) . . . 13

2.4.3 Training and Testing Times . . . 14

2.4.4 R^2 Score . . . 14

2.4.5 Position Error . . . 14
2.4.6 Orientation Error . . . 14
2.5 Software Architecture . . . 15
2.6 Version Control and Project Management . . . 16

3 Implementation and Validation 17
3.1 Software Implementation . . . 17
3.1.1 Implementing the Machine Learning Models . . . 17
3.1.2 Data Processing . . . 17
3.1.3 Creating Visualisations . . . 18
3.2 Project Workflow . . . 18
3.3 Validation . . . 19
3.3.1 Maximum and Minimum Predictions on the Training Set . . . 19
3.3.2 K-Fold Cross-Validation . . . 20
3.3.3 Other Methods of Validation . . . 21

4 Results, Evaluation and Discussion 22
4.1 Evaluation . . . 22
4.1.1 MAE Results . . . 22
4.1.2 MSE Results . . . 22
4.1.3 Training and Testing Times Results . . . 22
4.1.4 R^2 Score . . . 23
4.1.5 Position Error Comparison . . . 24
4.1.6 Orientation Error Comparison . . . 24
4.2 Discussion . . . 25
4.3 Conclusions . . . 27
4.4 Ideas for future work . . . 28

References 31

Appendices 34

A Self-appraisal 34
A.1 Critical self-evaluation . . . 34
A.2 Personal reflection and lessons learned . . . 35
A.3 Legal, social, ethical and professional issues . . . 35
A.3.1 Legal issues . . . 35
A.3.2 Social issues . . . 35
A.3.3 Ethical issues . . . 35
A.3.4 Professional issues . . . 36

B External Material 37

C Additional Information 38

Chapter 1

Introduction and Background Research

1.1 Introduction

This paper will look at inverse kinematics in-depth, what it is used for, and how it is calculated, and use different machine learning algorithms to solve the inverse kinematics problem before evaluating the results and seeing which algorithm produces the best result in terms of accuracy and speed.

1.2 Robotics

Robotics is the study of designing and developing robots, a machine capable of carrying out a set of complex instructions [30]. Many different disciplines contribute to the creation of robots, from software engineering to material engineering. Robots are designed to complete tasks that could be too dangerous for humans, such as exploring deep under the ocean, or tasks that are seen as boring or repetitive, such as assembling products. Robotics is an increasingly growing field, with advancements being made faster and faster, the most recent of which is shown by the reveal of Tesla's Optimus robot [38].

1.2.1 History of Robotics

The concept of robotics and autonomous systems have existed for an extremely long time, with the first creation of a mechanical bird that could fly by being propelled by steam, being made in 4th century BC by Archytas of Tarentum [33]. In 1206, Al-Jazari of Baghdad wrote a book titled "The Book of Knowledge of Ingenious Mechanical Devices", which contains details of 50 different mechanical devices, along with instructions to make them. He is known today as the Father of Robotics due to thinking of several concepts that are essential to the development of robots [2]. In 1495, Leonardo da Vinci had drawn a detailed diagram of a mechanical knight that would be able to sit, move its head, and wave its arms [26]. Throughout this period of history, small advancements were made in the field of robotics, allowing robots to perform small, menial tasks but not able to reason or think. This began to change in the 20th Century, as in 1912, Leonardo Torres Quevedo built the first robot able to play an endgame of chess with three pieces without any human guidance [42]. In the 1930s, the earliest iterations of industrial robots were being put to use in factories across the United States. These were designed after the human arm, with joints similar to that of the human shoulder, elbow, and wrist to replicate movements like pushing, pulling, and lifting. The first patent for a robot of this design was made by Willard V. Pollard in 1938, but were computationally expensive [29]. In the late 1930s, the first programmable computer would be invented by Konrad Zuse, which would ensure the foundation for a humanoid robot [21]. The first electronic autonomous robots that can perform complex behaviour was invented in the late 1940s by William Grey Walter, These robots, named Elmer and Elsie, were light-sensitive and touch-sensitive, which interacted

together to allow the robots to avoid obstacles [41]. In the 1950s, George Devol created the first digitally operable and programmable industrial robot, called Unimate, which helped to turn the modern robotics industry into what it is today [28]. In 1969, Victor Scheinman created the first electric-powered, computer controlled robot arm called the Stanford Arm. By 1974, it could assemble a Ford Model T water pump [7]. That same year, German-based company KUKA developed FAMULUS, the first industrial robot with six electromechanically driven axes [35]. In 1986, Honda began their robotics research, with the most advanced result of their research being revealed in 2005, named ASIMO. ASIMO is a humanoid robot that can run, walk, talk to people, recognise people and environments, and interact with it [16]. There were huge improvements made in the field of robotics in the 2010s, particularly when it comes the availability of robotics components, as well as the integration of robots into everyday life. This also led to many discussions around concerns whether robots could gain consciousness, and what should humanity's relationships with these robots look like. Now, robots are found commercially and in industry, being able to perform jobs more efficiently and accurately compared to humans.

1.2.2 The Role of Software in Robotics

Software has a large role to play in robotics, as it can impact various areas required for a robot to function properly. Various computer vision frameworks and techniques are designed to help robots process what they are seeing, and identify objects based on their shape and colour. Path-planning algorithms are implemented to allow robots to interact with their environment, and imitation and reinforcement learning are used to help robots to learn certain actions. Various computational advancements over the last couple of decades have taken robots from being able to perform basic tasks repeatedly to being able to learn and perform complex tasks by itself, such as the TOPIO robot being able to play table tennis against a human player [3].

One such advancement would be Artifical Intelligence (AI), which has seen huge advancements the last 5 years or so thanks to tools like OpenAI's ChatGPT and Microsoft's CoPilot. Perez et al, 2017 discusses how the implementation of AI in Robotics is important for allowing robots to be more autonomous, which can be broken down into the perception, planning, and execution of commands done by the robot [31].

1.3 Kinematics

Kinematics is the study of motion without needing to consider the moments or forces that cause the motion to occur [9]. In robotics, there are two forms of kinematics. There are forward kinematics, which finds the end position and orientation of an end-effector using its joint angles, and inverse kinematics, which finds the joint angles necessary for a given position and orientation of the end-effector (See Figure 1.1). For example, Figure 1.2 shows the position of a robot arm at point (a), with set values of θ_1, θ_2, and θ_3 for its corresponding joints J_1, J_2, and J_3. For the the arm to be in the position shown in (b), the new values for the joint angles are needed to be known.

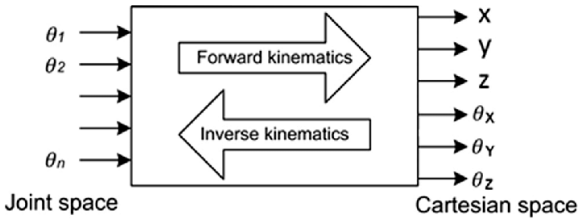


Figure 1.1: The difference between forward and inverse kinematics [37]

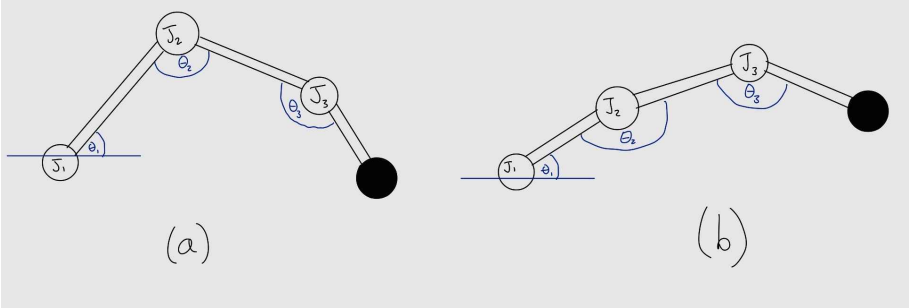


Figure 1.2: (a) Initial position of the end-effector; (b) Desired position of the end-effector

1.3.1 The Inverse Kinematics Problem

As said earlier, inverse kinematics is finding the joint angles of a manipulator given its desired pose. The position and orientation, represented by a vector and matrix respectfully, are in Cartesian space, whereas the joint angles for the joints in the manipulator are in joint space. The inverse kinematics problem comes down to being able to convert the position and orientation from Cartesian space to joint space [22].

1.3.2 Importance of Inverse Kinematics

Inverse kinematics is extremely important in robotics, as it allows the robot to make precise movements to the desired location. The end-effector of a robot will consist of multiple joints, all working together. Causing one joint to move will cause the entire position of the arm to be completely different, hence the need to be able to find the angle for each joint in order to find the desired position for the end-effector, even if it is a minute movement.

There are many applications for inverse kinematics. Within the field of Robotics, inverse kinematics is used to define precise movement, making it extremely useful for automated tasks to manipulate objects. Outside of robotics, inverse kinematics is useful within the field of computer graphics, particularly when it comes to animating subjects which have joints [1].

1.3.3 Closed-form Solutions of the Inverse Kinematics Problem

A closed-form solution is described by Craig, 2005 as a method based on analytical expressions [8]. The problem is set up below.

Consider the UR5 end-effector with 6 joints, shown in Figure 1.3. Each joint is labelled as J_1, J_2, J_3, J_4, J_5, and J_6, and the joint angles for each joint are labelled as θ_1, θ_2, θ_3, θ_4, θ_5, and θ_6.

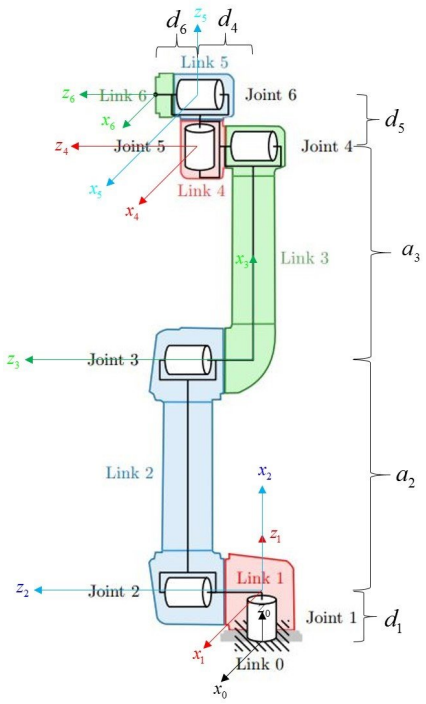


Figure 1.3: A diagram of the UR5 robot structure [18]

Each joint has a coordinate frame associated with it, and a transformation matrix representing the pose of another coordinate frame inside of it. For example, looking at the coordinate frame for J_1 and the frame for J_2 in Figure 1.3, the transformation matrix ${}^{J_1}T_{J_2}$ can be determined as:

$$ {}^{J_1}T_{J_2} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & {}^{J_1}\mathbf{X}_{J_2} \\ R_{21} & R_{22} & R_{23} & {}^{J_1}\mathbf{Y}_{J_2} \\ R_{31} & R_{32} & R_{33} & {}^{J_1}\mathbf{Z}_{J_2} \\ 0 & 0 & 0 & 1 \end{bmatrix} $$

where R is a 3D rotation matrix representing the orientation of the coordinate frame J_2 within J_1, and ${}^{J_1}\mathbf{X}_{J_2}$, ${}^{J_1}\mathbf{Y}_{J_2}$, and ${}^{J_1}\mathbf{Z}_{J_2}$ represents the origin of the coordinate frame J_2 within J_1. These transformation matrices can be chained together to find the transformation matrix representing the desired end-effector pose [8]:

$$ {}^{J_1}T_{J_6} = {}^{J_1}T_{J_2} \times {}^{J_2}T_{J_3} \times {}^{J_3}T_{J_4} \times {}^{J_4}T_{J_5} \times {}^{J_5}T_{J_6} $$

The inverse kinematics problem can be asked in terms of these matrices, i.e. given ${}^{J_1}T_{J_6}$, how does one find the joint angles that will give the desired position?

This can be formulated this as an optimisation problem. Let there be a 6×1 vector called θ that stores all the joint angles, and 3×1 vector called p that stores the end-effector position:

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \end{bmatrix} \qquad p = \begin{bmatrix} {}^{J_1}\mathbf{X}_{J_6} \\ {}^{J_1}\mathbf{Y}_{J_6} \\ {}^{J_1}\mathbf{Z}_{J_6} \end{bmatrix}$$

The smallest value of θ that minimises the distance between the current position of the end-effector and the target position is desired. There are many ways to find this value of θ, two of which are the geometric solution and the algebraic solution.

Geometric Solution

Craig, 2005 discusses how to find the geometric solution for an end-effector with 3 joints by splitting the spatial geometry of the arm into multiple plane-geometry problems, and solving each one for each joint angle. [8]

Algebraic Solution

Craig, 2005 discusses how to find the algebraic solution for an end-effector with 3 joints by creating a system of 4 non-linear equations that can be solved to find the necessary joint angles. [8]

1.3.4 Numerical Solutions of the Inverse Kinematics Problem

There are many different numerical methods for solving the inverse kinematics that use the Jacobian matrix. which is a function of the joint angles of an end-effector, shown below:

$$J = \begin{bmatrix} \frac{\delta p_x}{\delta \theta_1} & \frac{\delta p_x}{\delta \theta_2} & \cdots & \frac{\delta p_x}{\delta \theta_n} \\ \frac{\delta p_y}{\delta \theta_1} & \frac{\delta p_y}{\delta \theta_2} & \cdots & \frac{\delta p_y}{\delta \theta_n} \\ \frac{\delta p_z}{\delta \theta_1} & \frac{\delta p_z}{\delta \theta_2} & \cdots & \frac{\delta p_z}{\delta \theta_n} \end{bmatrix}$$

J relates the end-effector position in cartesian space, represented as p_x, p_y, and p_z, to however many joint angles there may be within the end-effector, represented as θ_1, θ_2, up to θ_n, where n is the number of joints in the end-effector, as first-order derivatives.

Jacobian Transpose Method

One of the first methods developed for solving the inverse kinematics problem using J was the Jacobian Transpose method. Buss, 2009 discusses how to use the transpose of the Jacobian matrix, represented at J^T, and multiplying it with a scalar value α and a vector $\vec{e}$ which represents the change from the current position to the desired position, to find out the vector $\Delta\theta$, which represents the minimum value that each joint angle is changed by to get to the target position. The transpose is gained by swapping the rows and columns of J, which can be seen below [5]:

$$J^T = \begin{bmatrix} \frac{\delta p_x}{\delta \theta_1} & \frac{\delta p_y}{\delta \theta_1} & \frac{\delta p_z}{\delta \theta_1} \\ \frac{\delta p_x}{\delta \theta_2} & \frac{\delta p_y}{\delta \theta_2} & \frac{\delta p_z}{\delta \theta_2} \\ \cdots & \cdots & \cdots \\ \frac{\delta p_x}{\delta \theta_n} & \frac{\delta p_y}{\delta \theta_n} & \frac{\delta p_z}{\delta \theta_n} \end{bmatrix}$$

Pseudoinverse Method

Buss, 2009 also discusses the Pseudoinverse method, which uses the pseudoinverse, also known as the Moore-Penrose inverse, of J, represented as J^+, and multiplying it with the vector $\vec{e}$ to find out $\Delta\theta$. To get the pseudoinverse of J, it depends on how big J is, and thus how many joints the end-effector has. If the end-effector has more than 3 joints, i.e. $n > 3$, then the equation to calculate J^+ is:

$$J^+ = (J^T J)^{-1} J^T$$

If the end-effector has less than 3 joints ($n < 3$), then the equation is [5]:

$$J^+ = J^T (J J^T)^{-1}$$

Damped Least Squares Method

There is also the Damped Least Squares Method, also known as the Levenberg-Marquardt method. Buss, 2009 shows how it is used to find $\Delta\theta$ using J and a non-zero damping constant $\lambda \in \mathbb{R}$. The solution can be written as:

$$\Delta\theta = J^T (J J^T + \lambda^2 I)^{-1} \vec{e}$$

where I is the identity matrix [5].

1.4 Quaternion Space

Most movement for an end-effector is defined in a coordinate space that is different from the space in which the motor commands occur, meaning that if a position is given in Cartesian coordinates, the rotation angles and translation of the end-effector will need to be represented using a different system. There are many systems that to choose from, one being the quaternions [9].

Quaternions are a mathematical concept founded by William R. Hamilton that is used to describe rotations in 3-dimensional space. They are a four-dimensional extension of the complex number system that has the form of the equation below:

$$Q = w + ix + jy + kz$$

where w, x, y, and z are members of the real set of numbers $\mathbb{R}$, and i, j, and k are members of the complex set of numbers $\mathbb{C}$ [13]. Q represents the rotation vector.

Since Hamilton's [13] discovery, quaternions have been used in a myriad of fields, from aerospace engineering to quantum mechanics. The first time quaternions were considered for the use of kinematics was in 1895 by Kotelnikov [20]. Funda et al. discusses how using a dual-quaternion system to represent rotations and translations can significantly improve storage efficiency and numerical robustness compared to using homogeneous transforms [12]. Quaternions were then used to solve the forward and inverse kinematics problem by Kim et al in 1990 for an end-effector with 6 degrees of freedom [19].

1.5 Machine Learning

Arthur Samuel defines Machine Learning as the ability of computers to learn without being explicitly programmed to learn [34]. Machine learning comes in many different formats, each with its own advantages and disadvantages. These branches can be seen above in Figure 1.4.

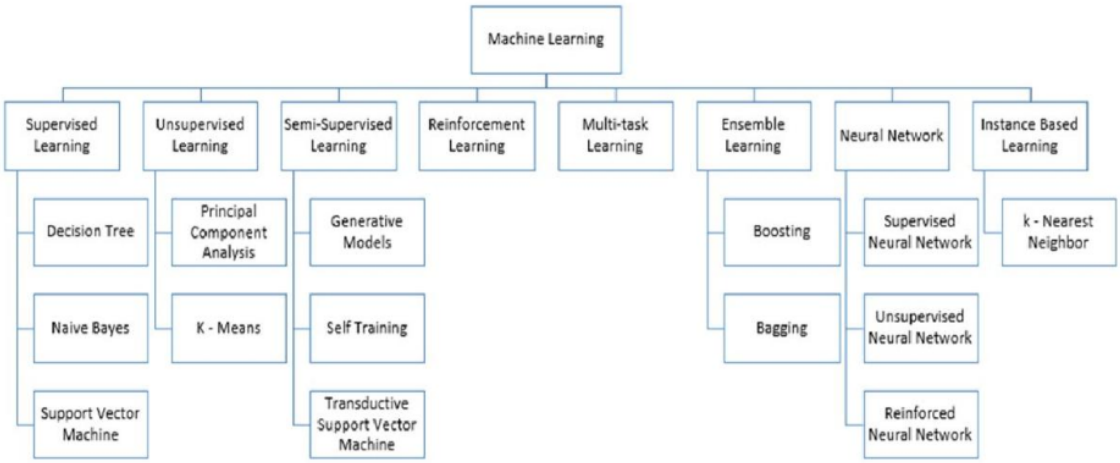


Figure 1.4: The branches of Machine Learning [23]

1.5.1 Supervised Learning Algorithms

Supervised learning algorithms are one of the major forms of machine learning algorithms. A supervised algorithm learns how a function works when taking in an input and producing an output by training itself on a training set of data consisting of input-output pairs. The algorithm learns the pattern from the training set and then applies this to the test set of data to predict what the output should be [23]. There are two main kinds of supervised learning algorithms, regression algorithms and classification algorithms

Regression Algorithms

A regression algorithm takes in continuous data to learn and predict the output of a function. For example, a function $f : X \to Y$ where X is the input of a person's height and predicts Y, their age [40]. Some examples of regression algorithms are Linear Regression, Gaussian Process Regression, and Lasso Regression.

Classification Algorithms

A classification algorithm takes in discrete data to learn and predict the output of a function [36]. For example, a function $f : X \to Y$ where X is the input of incoming emails and predicts Y, whether or not the emails are fraudulent. Some examples of classification algorithms are k-Nearest Neighbours and Decision Trees.

1.5.2 Ensemble Methods

Ensemble methods are another form of machine learning algorithms. They are algorithms that use the individual results of multiple different algorithms, typically classifier algorithms, and combine the results to predict the outputs for a given set [11]. Some examples of these methods are the Random Forest and Gradient Boosting algorithms.

Chapter 2

Methodology

2.1 Finding a Dataset

The first thing needed to start this project was a dataset consisting of end-effector positions and its corresponding joint angles. At first, this was quite difficult to find online, especially since knowing the robot's end-effector design was crucial in order to simulate the end-effector position correctly. With assistance from a supervisor, contact was made with a colleague who provided a pickle file. It has 11 different real trajectories, with each trajectory having having a different number of timesteps, and each timestep containing 10 values. The first 3 dimensions represent the end-effectors position (input X), and the remaining 7 dimensions represent the corresponding joint angles (output Y). The dataset is split such that 80% of the dataset is used for training the models, and the other 20% is used for testing.

2.1.1 Data Processing

The angles from the dataset were then encoded into their respective sine and cosine values and used to train the models rather than the raw angle values. This has several advantages, one being that it allows smoother transitions between values. Using raw angles to train the model may cause the models to think that there is a large difference between 0 degrees and 359 degrees, when there is no large difference. Training the models on their sine and cosine values ensures that this does not happen, and avoids any discontinuity. The sine and cosine values also capture the position of the joint angles in a 2D space, meaning that the models can learn the relationships found inside of the dataset without being affected by arbitrary scale differences in the raw values. To decode the angles, the $arctan2(sin(\theta), cos(\theta))$ function is used.

Once the angles were encoded, X and Y were scaled using a `StandardScaler` class from the `scikit-learn` library for each X and Y. This is to standardise the input to have zero mean and unit variance (i.e. the standard deviation becomes 1), which prevents any features found in the dataset to take over the model's learning process and impact it. Separate scalers were used for the end-effector position and the joint angles respectively as they have different units and distribution, so it ensures that they are standardised independently of each other. The X Scaler is applied as part of a pipeline, discussed later in this chapter. The Decision Tree, Random Forest, and Gradient Boosting models do not use the scaled data as they are tree-based models that split data based on feature values rather than absolute values (Discussed further later in this chapter).

2.2 Finding a Suitable Robotics Simulator

Once the dataset was collected, the next task was to find a simulator that would be able to simulate the different end-effector poses. There were many different simulators to choose, from

ROS to Webots. After researching, the PyBullet library was found.

2.2.1 PyBullet

The PyBullet library is a Python module designed for physics and robotic simulation. This was a perfect choice for this project as it could simulate the different end-effector poses created when testing the different machine learning models, and contained functions to calculate the inverse kinematics problem. A search was conducted to look for a robotics simulator that uses this library. There were many available on GitHub, but one that had included inverse kinematic calculations using the built-in inverse kinematics solver from PyBullet, titled `pybullet-kinematics-dynamics-control`, was chosen for this project, being stored as a Git submodule.

2.2.2 Robotic End-Effector Design

The design of the end-effector was also needed to be known in order to simulate the predicted angles within the simulator. The robot end-effector used to generate the data is a Franka Emika Panda model with 7 joints, or Degrees of Freedom (DOF) (See Figure 2.1). The corresponding URDF (Unified Robot Description Format) file was needed to implement this end-effector design into the simulator. After looking online, one available on GitHub called `PandaRobot.jl` was used for this project, being stored as a Git submodule.

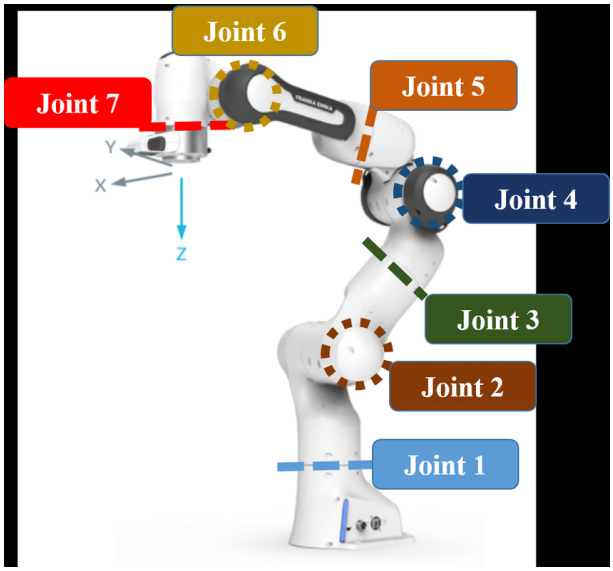


Figure 2.1: Franka Emika Panda Model with 7 Joints [15]

2.3 Choice of Machine Learning Models

Once the dataset and the simulator were set up, the next task was to choose which Machine Learning models to implement. Not every single model could be tested as the problem lent itself to being solved by supervised models, due to the data being labelled and in the form of $f : X \to Y$. There are many different models to choose from, from kNN to Decision Trees and more. The models chosen are:

2.3.1 k-Nearest Neighbours (kNN)

The kNN model is a classification algorithm that classifies data based on the class of its nearest neighbour (or k nearest neighbours, as it can be based on the classes of multiple neighbours) [10]. For this project, kNN is being used as a regression algorithm by finding the average of the outputs of its neighbours. When given a new end-effector position, kNN searches for the k closest neighbors in the dataset using a distance metric (e.g., Euclidean distance in Cartesian space) to calculate the predicted joint angles.

2.3.2 Linear Regression

The Linear Regression algorithm models a relationship between two variables, assuming there is a linear relationship between them using the Ordinary Least Squares (OLS) method [17]. For this project, Linear Regression is being used as it is a very straightforward approach, and can easily model a linear relationship between the end-effector position and the corresponding joint angles.

2.3.3 Neural Network

The Neural Network model is an algorithm that can be used for both regression and classification. It is designed to simulate the structure and function of biological neural networks inside of the human brain by applying linear functions followed by non-linear functions [25]. For this project, a Deep Neural Network is being used, which allows the model to have multiple hidden layers so the model can learn any complex, non-linear relationships there might exist between the end-effector position and the corresponding joint angles.

2.3.4 Decision Trees

The Decision Tree model can be used for regression and classification. The model recursively splits the dataset based on features to form a tree-like structure that contains choices at each node until a stopping criteria is met. The algorithm decides which feature to use for each split based on data like Information Gain or the Gini Index [24]. For this project, a Decision Tree regression model is used, as it learns from the training set to create the tree structure which is then used to find and predict the solutions for the testing set.

2.3.5 Support Vector Regression (SVR)

The SVR algorithm is based on the principles of Supprt Vector Machines. It is designed to predict continuous values and finds the best approximation for the relationship between the input and output within a given tolerance [39]. By training this model on the training set, the SVR model can find a function to predict the new joint angles of the testing set.

2.3.6 Random Forest

The Random Forest algorithm is an ensemble method that combines multiple Decision Tree models to predict the output of a given dataset for both regression and classification. The Decision Trees are built over different subsets of the dataset and the average of their output is

taken to make the final predictions [4]. For this project, the Random Forest model is being used as a regression model to capture non-linear relationships between the end-effector position and the corresponding joint angles, whilst also possibly providing a better solution to just using a singular Decision Tree.

2.3.7 Gradient Boosting

The Gradient Boosting model is an ensemble method that uses multiple Decision Trees like the Random Forest model. The difference between them is that the Gradient Boosting model builds multiple weak Decision Trees sequentially, meaning that the newest tree corrects any errors found in the previous tree using gradient descent [14]. While it can be used for both regression and classification, the Gradient Boosting model will be used for regression in this project, as it will be able to refine its predictions by learning from its errors, meaning it should be able to capture any non-linear relationships found between the end-effector position and the corresponding joint angles.

2.3.8 Gaussian Process Regression (GPR)

The GPR algorithm models functions as a multivariate Gaussian distribution rather than assuming a fixed functional form. It is able to perform uncertainty estimation by not only providing predictions, but also confidence intervals for the predictions made [32]. For this project, the uncertainty estimations created by this model would show which predictions are precise when approximating the inverse kinematics problem.

2.3.9 Bayesian Linear Regression (BLR)

The BLR model is similar to the Linear Regression model. The difference is that the Bayesian Linear Regression model doesn't use the OLS method, but rather Bayesian inference for each model parameter, treating them as probability distributions [27]. For this project, it can not only try and provide a relationship between the end-effector position and their joint angles, but can also provide confidence intervals for each parameter, and could perhaps perform better than the standard Linear Regression Model.

2.3.10 Lasso Regression

The Lasso Regression algorithm, also known as the L1 Regularisation, is similar to Linear Regression, except it applies a penalty to reduce overfitting and encourage sparse feature selection. The penalty applied is the absolute value of the coefficients of the model [14]. For this project, the Lasso Regression model can reduce overfitting when solving the inverse kinematics problem, and improve interpretability.

2.3.11 Ridge Regression

The Ridge Regression algorithm, also known as the L2 Regularisation, applies a penalty to reduce overfitting, just like the Lasso Regression model. The penalty for this model however, is the squared value of the coefficients of the model [14]. For this project, the Ridge Regression

model can reduce overfitting just like the Lasso Regression model, but how it does and whether it is better than the Lasso model or not is something that will be looked at.

2.3.12 Kernel Ridge Regression (KRR)

The KRR algorithm is an extension of the Ridge Regression algorithm, using kernel functions on top of the Ridge model to model complex, non-linear relationships [14]. For this project, the Kernel Ridge Regression model can learn any complex relationships found between the end-effector position and the corresponding joint angles whilst also potentially reduce overfitting.

The project has twelve different supervised regression models to use to solve the inverse kinematics problem and evaluate their performance.

2.4 Metrics Used to Evaluate Models

There are many ways to evaluate how well a model can be performing in terms of speed and accuracy. For this project, the following metrics were primarily focused on:

2.4.1 Mean Absolute Error (MAE)

The MAE is a measure of the average absolute differences between the predicted values and the actual values. It is defined as:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and n is the total number of data points.

MAE provides a measure of prediction error by expressing it in the same units as the target variable (in this case, the units would be in radians). It is useful because it treats all errors equally, making it less sensitive to outliers than squared error metrics. Lower MAE values indicate better model performance.

2.4.2 Mean Squared Error (MSE)

The MSE is a metric that is very similar to the MAE, but measures the average squared differences between predicted and actual values rather than the absolute values, shown in the equation below:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

The MSE penalises larger errors more than smaller ones due to the squaring operation, making it particularly useful when large deviations need to be minimized. However, this also means that MSE is more sensitive to outliers the the MAE. A lower MSE indicates better model performance. The units for the MSE is radians^2

2.4.3 Training and Testing Times

The training time is the time taken to train the model on the training dataset, and the testing time is the time taken to make predictions on the testing dataset. A model with a small training time is more appropriate for iterative experimentation and parameter tuning, whereas a model with a small testing time is crucial for real-time robotics applications where quick decision-making is necessary. Having a small training and testing time would make a model successful in both these areas.

2.4.4 R^2 Score

The R^2 Score, also known as the Coefficient of Determination, is a unitless metric that indicates how good the fit of the data is to the predictions created by the model. It is especially useful for determining the overall fitness of regression models. It is provided as:

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the actual values [17].

A value of 1 for the R^2 score represents a perfect fit, and a value of 0 signifies that the predictive capability of the model is the same as predicting the target variable's mean. A negative value shows that the model is worse than the mean prediction.

2.4.5 Position Error

The position error is a measure of the difference between the predicted end-effector position and the actual position in 3D space. It is extremely crucial in inverse kinematics, as the accuracy of arriving at the target position is a key requirement.

Mathematically, position error is most commonly computed using the Euclidean distance:

$$E_{pos} = |p_{pred} - p_{true}|$$

where p_{pred} is the predicted position vector and p_{true} is the actual position vector. This is what has chosen to be used for this project.

Small position error values reflect improved inverse kinematics solutions, thereby guaranteeing that the robot's end-effector accurately reaches the target. This metric is measured in metres.

2.4.6 Orientation Error

The orientation error is a measure of the difference between the predicted end-effector orientation is from the actual orientation. Orientation in robotics is typically expressed in rotation matrices, quaternions, or Euler angles as explained in Chapter 1.

A standard method of estimating the orientation error is to calculate the angular difference between the predicted and true rotation quaternions:

$$E_{ori} = \cos^{-1}(|q_{pred} \cdot q_{true}|)$$

where q_{pred} and q_{true} are unit quaternions representing the predicted and actual orientations, respectively [6]. This is what has been chosen to use for this project.

Minimization of orientation error is of utmost significance in applications that demand accurate positioning of the end-effector, including robotic assembly and surgical robotics. This metric is measured in radians.

2.5 Software Architecture

The code has the following structure:

```
inverse_kinematics_sim/
├── generate/
│   └── generate_data.py
├── models/
│   ├── decision_trees.py
│   ├── gaussian_regression.py
│   ├── gradient_boosting.py
│   ├── kernel_ridge.py
│   ├── kNN.py
│   ├── lasso.py
│   ├── linear_regression.py
│   ├── neural_networks.py
│   ├── random_forest.py
│   ├── ridge.py
│   └── svr.py
├── pybullet_kinematics_dynamics_control/
│   ├── pybullet_controller.py
│   └── ...
├── results/
│   ├── plot.py
│   └── ...
├── urdf/
│   └── PandaRobot.jl/
│       └── ...
├── main.py
├── requirements.txt
└── robot-trajectory.pkl
```

The `generate_data.py` file contains functions for encoding and decoding the angles, calculating the position and orientation errors, as well as a function to perform the model testing.

The `models` folder contains files for each of the different machine learning models implemented (the Linear Regression model and Bayesian Linear Regression are in `linear_regression.py`). Each of the models creates a pipeline using the `make_pipeline` function and performs hyperparameter tuning (discussed in Chapter 3) before testing the model and finding the

metrics discussed above.

The `pybullet_kinematics_dynamics_control` folder is the GitHub repository that has the simulator used for this project. It requires the URDF file to be stored inside a `urdf` folder, hence why the `PandaRobot.jl` submodule is inside of the `urdf` folder.

The `plot.py` file contains the functions for generating the graphs and tables of the results, which is then stored inside of the `results` folder.

The `main.py` file brings everything together, first reading the data from the `robot-trajectory.pkl` pickle file, and then encoding and scaling the data, then running the training and testing for each model before running the plotting functions.

The `requirements.txt` file stores all the required packages necessary to build and run the program.

2.6 Version Control and Project Management

In order to ensure the code was secure, accessible, and having the ability to control the current version of the project, Git was used to push the code and any changes to a remote repository on GitHub. Commits were made detailing the changes that were made since the last commit.

A Kanban Board was also used to ensure the project was managed efficiently and effectively. It kept track of a list of what tasks needed to be undertaken, keep track of their progress, and write out any findings or issues found with the current task (See Figure 2.2).

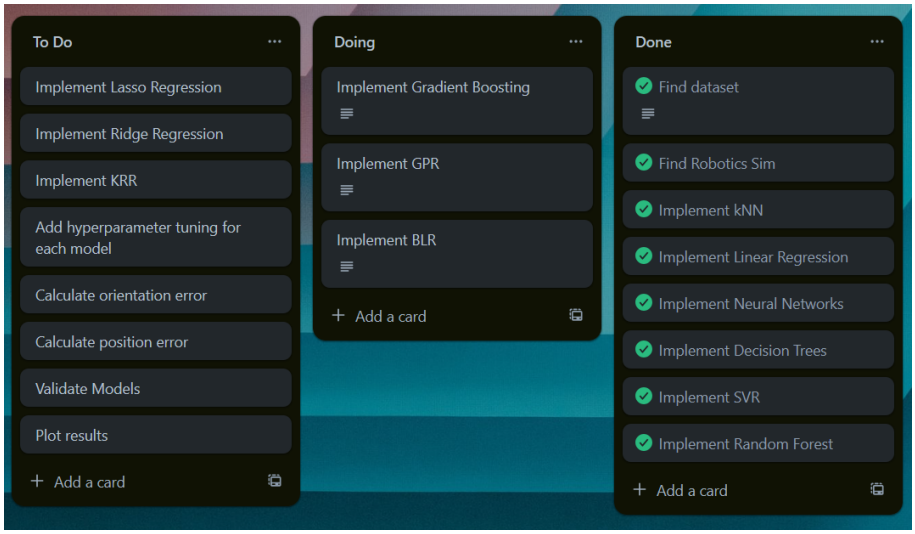


Figure 2.2: Kanban Board used for Project Management

Chapter 3

Implementation and Validation

3.1 Software Implementation

The project was implemented using the Python language, as the robot simulator chosen was written in Python, and the language also has access to many libraries for processing data, creating the machine learning models, and creating the graphs.

3.1.1 Implementing the Machine Learning Models

Each of the machine learning models were implemented using the `scikit-learn` library. This is because it contains the classes for each of the models chosen for this project, whilst also having functions that can split the dataset into a training set and testing set, and scale the data appropriately, as discussed in Chapter 2.1.1.

Hyperparameter Tuning

For each of these models, hyperparameter tuning was implemented. Each of these models has different parameters that can be changed to produce different results, e.g. the number of neighbours (k) in the kNN algorithm. To ensure each model was producing the best results, a dictionary of the parameters for each model was made, containing what the possible values could be. The `GridSearchCV` function from the `Scikit-Learn` library was used to try out every possible combination of parameters to find the best parameters for the models. The `GridSearchCV` function has a parameter called `scoring`, which tells the function to choose the parameter which has the best score for a certain metric. The MSE value was chosen as the scoring metric as it penalises larger errors, which could potentially lead to significant position and orientation errors. Using the MSE as the scoring metric ensures that the best model is the one that doesn't lead to large angle errors. The Gradient Boosting and GPR algorithms use the `RandomizedSearchCV` function instead, which is similar to the `GridSearchCV` except it samples parameters randomly from a distribution. This is used to reduce the time taken to tune both of the models and find their optimal parameters much more quickly, as the time taken to train these models were very sensitive to the size of the dataset.

3.1.2 Data Processing

For the preprocessing and postprocessing of the data, the `numpy` library was used. It had the sine and cosine functions for encoding the angles, as well as the arctan2 function for decoding them, and the functions to carry out the the calculations for the position and orientation errors. The `scikit-learn` library also has the class to scale the input and output data.

3.1.3 Creating Visualisations

To create the visualisations, the `matplotlib`, `pandas` and `seaborn` libraries were all used. The `matplotlib` library was essential to create the graphs necessary to visualise the metric values each model got, e.g. the MAE value for each model (See Figure 4.1). The `pandas` library was needed to create the tables of the metric scores for each model, which was then used by the `seaborn` library to create the violin plots displaying the position and orientation error.

Implementing a pipeline that collects and processes data, trains and tests twelve different machine learning models, performs hyperparameter tuning to identify the optimal version of each model, calculates evaluation metrics, and produces visualizations introduces significant challenges. To ensure success, the problem was broken down into smaller problems, tackling them step by step — such as handling the scaling and encoding/decoding of joint angles individually. Ensuring compatibility across all models and consistency in their outputs added further complexity, but this was addressed by using modular, reusable code with robust exception handling to ensure the models operated as expected.

3.2 Project Workflow

The project first starts by creating an instance of the robot simulator, ensuring it is pointing to the correct URDF file. The project then loads the dataset from the pickle file, and the joint angles from the dataset are then encoded into their sine and cosine pairs. The dataset is then split into a training and testing set. Once done, the encoded joint angles are scaled using the `StandardScaler()` class, and then the datasets are passed to the models for training. The model workflow can be seen in Figure 3.1.

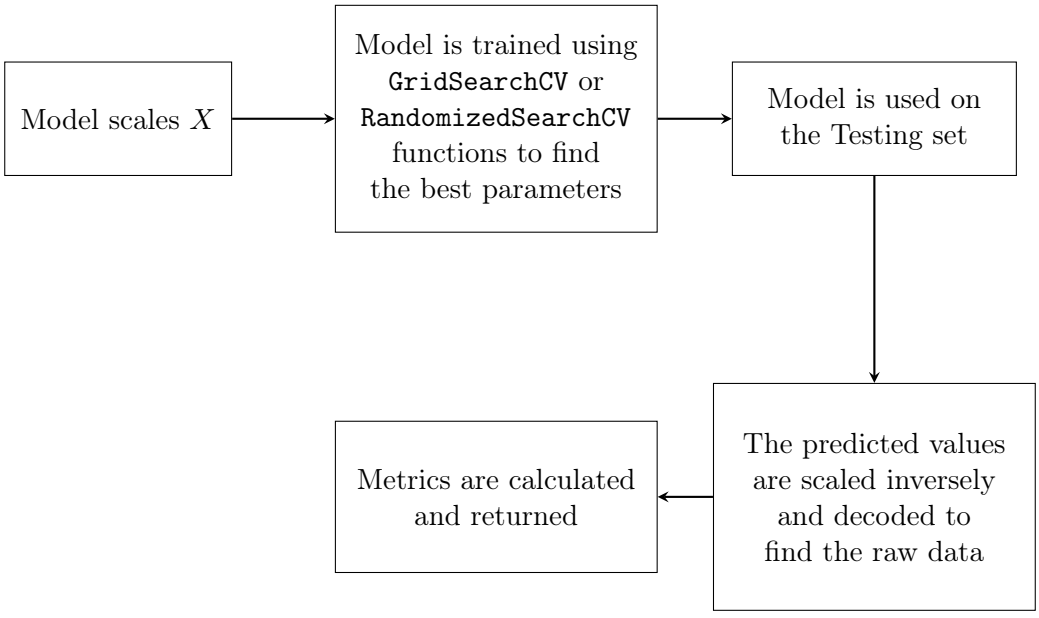


Figure 3.1: Machine Learning Workflow

The model creates a pipeline using the `make_pipeline()` function, which is useful for applying

the scaler for the X data (the end-effector position) and then perform the training for the model. Once the pipeline is made, a dictionary of parameters is made, which is then used to perform hyperparameter tuning. Once the tuning is complete, the optimal model is found, as well as the time taken to train that specific model, and use it on the testing set. After predicting the corresponding Y values, the scaler used previously for the joint angles is used again to inversely scale them back. Once done, the angles are decoded and returned back. The testing time is also measured during this process. Once the raw predicted values are found, the metrics are calculated against the raw test values. The model then returns the values of all the metrics, as well as the best parameters chosen.

Once all the models have been trained, tested, and had the metrics calculated and returned, the project creates the graphs and tables to represent this data.

3.3 Validation

The following tests are reproducible, as the dataset never changes, and the hyperparameter tuning will always find the best version of the model, which will always be the same everytime the project is ran.

3.3.1 Maximum and Minimum Predictions on the Training Set

In order to ensure the models were working as expected, each model was evaluated on the training set. The maximum and minimum predicted joint angles for each model were found, and compared against the true maximum and minimum joint angles of the training set. The absolute error between them was found, and compared against a tolerance value of $\epsilon = 0.05$ to see whether the predictions were within an acceptable range. The results of this test can be seen in Tables 3.1 and 3.2. The values in red represent values that have a margin of error greater than the tolerance value. The units of the joint angles are in radians.

Models	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7
Y (Training Set)	-0.622	-0.691	-0.279	-2.871	-0.269	1.836	-0.672
kNN	-0.622	-0.691	-0.279	-2.871	-0.269	1.836	-0.672
Linear Regression	-0.422	-0.573	-0.157	-2.811	-0.097	1.997	-0.028
Neural Networks	-0.658	-0.692	-0.288	-2.897	-0.312	1.825	-0.716
Decision Trees	-0.622	-0.691	-0.279	-2.872	-0.269	1.834	-0.672
SVR	-0.412	-0.493	-0.146	-2.806	-0.089	2.019	-0.034
Random Forest	-0.622	-0.691	-0.279	-2.872	-0.269	1.836	-0.672
Gradient Boosting	-0.642	-0.691	-0.291	-2.873	-0.306	1.812	-0.686
GPR	-0.626	-0.691	-0.282	-2.872	-0.282	1.835	-0.674
BLR	-0.421	-0.572	-0.156	-2.809	-0.098	1.996	-0.029
Lasso Regression	-0.418	-0.560	-0.147	-2.804	-0.095	2.010	-0.019
Ridge Regression	-0.421	-0.571	-0.156	-2.809	-0.098	1.996	-0.029
KRR	-0.686	-0.693	-0.291	-2.879	-0.315	1.831	-0.719

Table 3.1: Minimum Predictions against Minimum Training Values ($\epsilon = 0.05$)

Models	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7
Y (Training Set)	0.214	1.015	0.699	-0.884	0.317	2.621	2.750
kNN	0.214	1.015	0.699	-0.884	0.317	2.621	2.750
Linear Regression	0.196	0.871	0.465	-1.303	0.205	2.573	1.904
Neural Networks	0.231	1.024	0.723	-0.865	0.338	2.670	2.855
Decision Trees	0.213	1.015	0.699	-0.884	0.317	2.621	2.750
SVR	0.193	0.856	0.453	-1.243	0.191	2.570	1.909
Random Forest	0.213	1.015	0.699	-0.884	0.317	2.621	2.750
Gradient Boosting	0.239	1.029	0.709	-0.855	0.323	2.645	2.791
GPR	0.228	1.017	0.699	-0.879	0.319	2.634	2.756
BLR	0.197	0.871	0.466	-1.303	0.204	2.572	1.904
Lasso Regression	0.189	0.871	0.466	-1.303	0.204	2.572	1.904
Ridge Regression	0.197	0.871	0.466	-1.303	0.204	2.572	1.905
KRR	0.239	1.019	0.704	-0.869	0.330	2.653	2.801

Table 3.2: Maximum Predictions against Maximum Training Values ($\epsilon = 0.05$)

As shown in both tables, most of the models have similar predicted values to the actual value of each joint angle, apart from the Linear Regression, SVR, BLR, Lasso, and Ridge Regression models. However, this behaviour is expected due to the way these models work, and their predictions are within the acceptable bounds to be evaluated, and will be discussed further in Chapter 4.

3.3.2 K-Fold Cross-Validation

Another way of validating that the models were working as expected was by running the K-Fold Cross-Validation test, which is a common technique used to assess a machine learning model's ability to train on unseen data. The procedure works by splitting the dataset into K equal sized parts. For each partition, a partition is chosen as the validation set, and the other $K - 1$ partitions are used as the training set. The model is trained on the training set, and evaluated on the validation set. The MSE value is found, and then the process repeats [14]. The process then finds the mean and standard deviation of the MSE score. For this project, the K-Fold Cross-Validation process was used on the training dataset with $k = 5$, and a tolerance value $\epsilon = 0.01$ for the mean MSE and a tolerance value of $\epsilon = 0.002$ for the standard deviation. The results of this process can be seen in Table 3.3, rounded to 4 decimal places.

The values in red represent values that are greater than the tolerance value. The Table shows that most of these models are under the tolerance value apart from the linear models - Linear Regression, SVR, BLR, Lasso Regression, and Ridge Regression - just like above. This is again due to the way these models work as they try to capture a non-linear relationship using a linear method. This is discussed further in the next chapter, as the values produced are within acceptable bounds.

Models	Mean MSE (radians^2) ($\epsilon = 0.01$)	STD MSE (radians^2) ($\epsilon = 0.002$)
kNN	0.0028	0.0004
Linear Regression	0.0879	0.0023
Neural Networks	0.0026	0.0004
Decision Trees	0.0086	0.0029
SVR	0.0873	0.0022
Random Forest	0.0043	0.0012
Gradient Boosting	0.0052	0.0001
GPR	0.0023	0.0005
BLR	0.0879	0.0023
Lasso Regression	0.0882	0.0023
Ridge Regression	0.0879	0.0023
KRR	0.0060	0.0012

Table 3.3: K-Fold Cross-Validation Results $k = 5$

3.3.3 Other Methods of Validation

There are other ways to validate these models to ensure robustness and correctness. One of which would be to calculate the forward kinematics of the end-effector using the predicted joint angles, and comparing the true ground position with the new calculated position. This inverse consistency check is a way of ensuring that the models maintain consistency by identifying any major discrepancies between the joint angles and end-effector positions. Although this technique has not been implemented within the project, it is an important step to take this project further to ensure the models perform reliably and are robust.

Chapter 4

Results, Evaluation and Discussion

4.1 Evaluation

Table 4.1 (found on page 28) shows the best parameters for each of the models from running the hyperparameter tuning, and Table 4.2 (found on page 29) shows the results of the MAE, MSE, and the R^2 Score for each of the models, all rounded to 3 decimal points.

4.1.1 MAE Results

Figure 4.1 shows the Mean Absolute Error found for each of the models. As seen on the graph, and in Table 4.2, the kNN model has the lowest MAE value of 0.007 radians, followed by the GPR model with a value of 0.009 radians. The SVR model has the largest MAE value of 0.163 radians, followed closely by the Linear Regression, BLR, Lasso Regression, and Ridge Regression models with each of them having an MAE value of 0.162 radians.

4.1.2 MSE Results

Figure 4.2 shows the Mean Squared Error found for each of the models. As seen on the graph, and in Table 4.2, the GPR model both have the lowest MSE value of 0.002 radians^2, followed by the kNN and Neural Network model with a value of 0.004 radians^2. The Lasso Regression model has the largest MSE value of 0.086 radians^2, followed closely by the Linear Regression, BLR, and Ridge Regression models.

4.1.3 Training and Testing Times Results

Figure 4.3 shows the time taken to train the model vs the time taken for the model to solve the inverse kinematics problem on the testing set, as well as in Table 4.3 (found on page 29), with the timings rounded to 3 decimal places.

As can be seen, the model with the fastest training time is the Linear Regression model, taking only 0.003 seconds, followed closely by the Ridge Regression model with a training time of

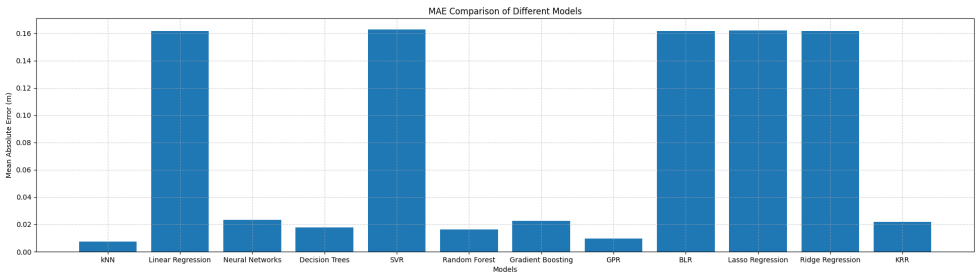


Figure 4.1: MAE Comparison of Different Models

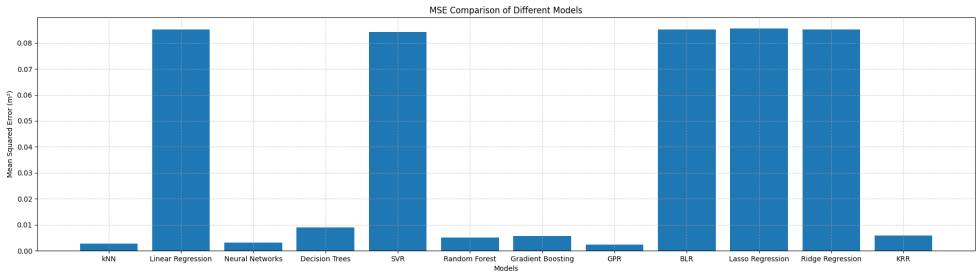


Figure 4.2: MSE Comparison of Different Models

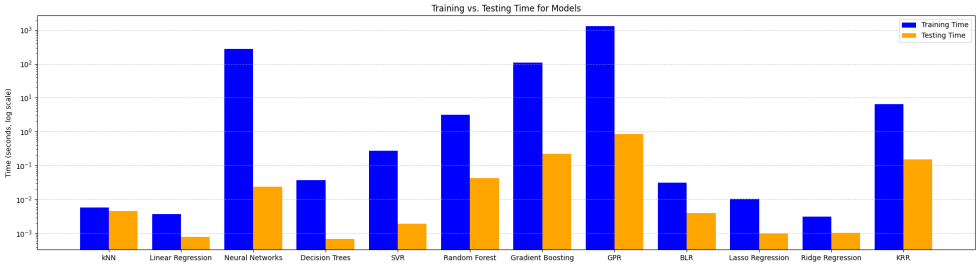


Figure 4.3: Timings Results of Different Models

0.005 seconds. The algorithm with the slowest training time is the GPR model, taking just under 20 minutes to be trained, followed by the Neural Network model which takes under 5 minutes to be trained.

The models with the fastest testing time are the Linear Regression, Decision Tree, Ridge Regression and Lasso Regression model, with them taking just a thousandth of a second to be trained. They are followed closely by the SVR and BLR models. The models with the slowest testing time is again the GPR model, taking 0.808 seconds to be tested, followed by the Gradient Boosting model.

4.1.4 R^2 Score

Figure 4.4 shows the R^2 score for each model. From reading this Figure and Table 4.2, the model with the largest R^2 score is the GPR model, with a near perfect score of 0.988, followed closely by the kNN model. The model with the smallest R^2 score is the SVR model with a score of 0.613, followed by the Lasso Regression model.

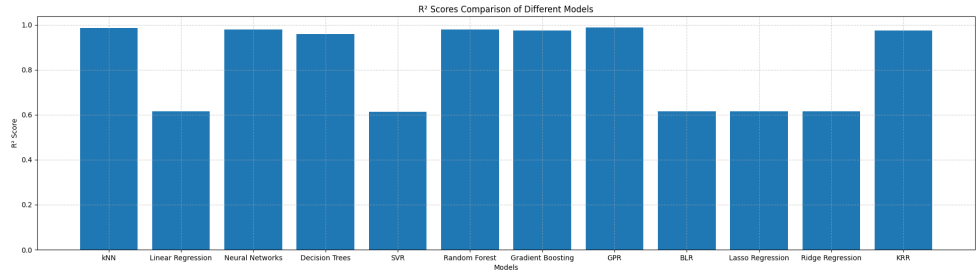


Figure 4.4: R^2 Score Comparison of Different Models

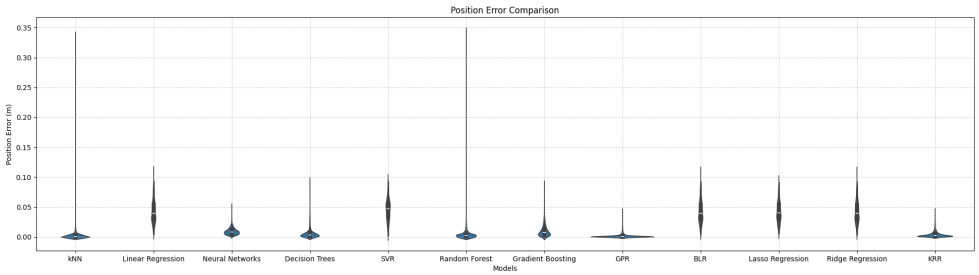


Figure 4.5: Position Error Results

4.1.5 Position Error Comparison

Figure 4.5 shows the position error distribution for each model as a violin plot. As stated in Chapter 2, the position error is calculated using the Euclidean Distance method. From this graph, the best performing models are the GPR model and the KRR model, followed closely by the Decision Tree and Neural Network models. This can be said because these are the models that have the least amount of variation (i.e. the plots are wide, not narrow). They also have the lowest median and interquartile range of errors, reaching near zero. The worst performing models are the Linear Regression, BLR, Ridge Regression, SVR, and Lasso Regression models, as these are the models with plots that tall and narrow, indicating a large amount of variation, as well as having the highest median error of around 0.04m.

An interesting thing to note is that the kNN and Random Forest models have a very wide plot each and low median errors, but they produce a maximum position error that is much greater than the other maximum position error of the other models. The maximum position error from these two models is greater than 0.325m, whereas the maximum position error from the other algorithms lays around 0.1m. This could be due to an outlier found within the dataset, which the kNN model and Random Forest models are both very sensitive to.

4.1.6 Orientation Error Comparison

Figure 4.6 shows the orientation error distribution for each model as a violin plot. As stated in Chapter 2, the orientation error is found by calculating the angular difference between the predicted and actual rotation quaternions. From this graph, the best performing model appears to be the kNN and GPR models, followed closely by the Neural Network and Random Forest models. The worst performing models are all the linear models - Linear Regression, SVR, BLR, Lasso Regression, and Ridge Regression.

One thing to note is that each of the models achieves a maximum orientation error that is roughly 1.7 radians or greater, despite how well it performs. This could suggest that there perhaps is an anomaly or an outlier found within the data, especially since the models that have the largest variation (the worst performing models mentioned above) seem to have the lowest maximum error whilst the best performing models seem to have the largest maximum orientation error.

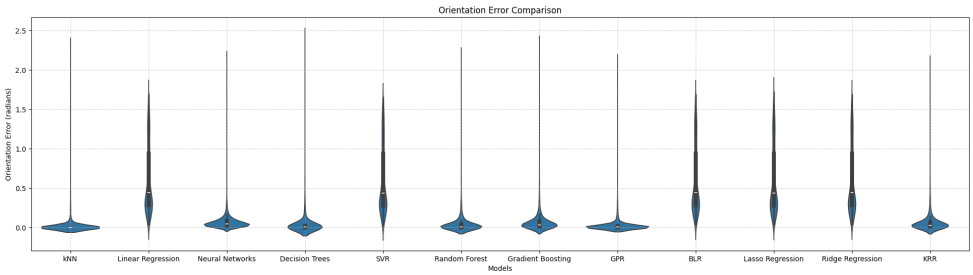


Figure 4.6: Orientation Error Results

4.2 Discussion

As seen from all of the results collected, the kNN model seems to be one of the best-performing models across all categories. It has one of the lowest MAE and MSE values, one of the highest R^2 scores, achieving a 0.985, and has one of the fastest times for being trained and tested, taking a total time of 0.011 seconds. The position and orientation error distributions are also tight and narrow as shown in the violin plots in Figures 4.5 and 4.6, meaning that the model is consistent, and the interquartile range of errors as well as the median error for both distributions are near zero. This could be because kNN is a non-parametric method, meaning it does not have a fixed form, but rather changes and grows in complexity as it sees more data. This means it is able to capture any complex, non-linear relationships found between the joint angles and the desired end-effector position effectively. However, the algorithm is sensitive to outliers, as shown in Figure 4.5, where it produced an extremely high maximum position error compared to the other models

The GPR model also has very good results, achieving extremely low values for the MAE and MSE, and achieving the highest R^2 score out of all the models. Like the kNN model, the GPR model is also very consistent, with tight, compact violin plots for the position and orientation error distributions. However, whilst the model performs very well, it lacks in timing. As stated aboved, the time taken to train the model is just over 20 minutes, which is impractical in a real-life scenario. This is because the GPR algorithm has a time complexity of $O(n^3)$, meaning that as the size of the dataset increases, the time taken to train the model will scale at a cubic rate. Given that the training dataset consists of just over 8,000 points, the training time will take a very long time.

The Random Forest and Gradient Boosting algorithms have also achieved great results, with a low MAE and MSE, and a high R^2 score of around 0.978 each. The training and testing time for the Random Forest model is just over 3 seconds, which when compared to other models in the list, is slow. The Gradient Boosting model is even slower, taking under 2 minutes to be trained and tested. The position error distributions also show that the models are effective, only having slightly more variation in the errors calculated than the GPR and kNN models. The Random Forest model seems to produce lower errors and have less of a spread compared to the Gradient Boosting model, which suggests that the Random Forest model is more consistent, and therefore better out of the two. However, just like with the kNN model, the

Random Forest algorithm produced an extremely high maximum position error, suggesting it is more sensitive to outliers compared to the Gradient Boosting model. The interesting thing to note, however, is that these two algorithms are ensemble methods as discussed in chapter 2.3.6 and 2.3.7. They use multiple Decision Trees in different ways to create their predictions. Therefore, one would think that both of these models would perform considerably better than just the use of a single Decision Tree. However, that doesn't appear to be the case. The Decision Tree model actually has a lower MAE score than the Gradient Boosting model, and is only 0.001 radians greater than the MAE for the Random Forest model. It has a slightly higher MSE than both the ensemble algorithms. The R^2 score does have a considerable gap, with a percentage difference of around 2% between the Decision Tree algorithm and the ensemble algorithms, but still well-performing when looking at the other models. The position error distribution shows the Decision Tree model is consistent, with it having less variation compared to the Gradient Boosting model, and the plot is very similar to the Random Forest model. The orientation error distributions for the Decision Tree model also show that the it is consistent like the other two models, with only a slight more variation. The Decision Tree algorithm is much faster than both the ensemble methods, not even taking a tenth of a second to be trained and tested. It can therefore be argued that the Decision Tree model performs better than the Gradient Boosting model, and has a similar performance to the Random Forest model. This unexpected difference in performance between the ensemble algorithms and the Decision Tree algorithm could be due to multiple reasons. Perhaps the inverse kinematics problem was already quite easy for the Decision Tree to solve, with not much variance between the training and the testing sets, or perhaps there was overfitting from the ensemble methods. This shows that perhaps the use of ensemble algorithms are not that much more efficient than the use of just a single algorithm for the use of solving the inverse kinematics problem.

Another result of note would be the performance of the Neural Network model. It has a strong set of results, with low MAE and MSE values, and a high R^2 score. However, it lacks in timings, having the second highest training time of 5 minutes. The position and orientation error distributions seem to have slightly more variation to those mentioned above. The Neural Network model didn't perform better than the non-parametric or tree-based models mentioned above, which could be due to a number of reasons, possibly overfitting, or the selection of parameters for the hyperparameter tuning were too simple.

Another result to note would be the difference in performance between the Ridge Regression and the Kernel Ridge Regression algorithms. Since the KRR model is simply an extension of the Ridge Regression model, as stated in 2.3.12, one would expect that the KRR model would outperform the Ridge model, which is exactly what is seen with the results. The KRR algorithm has an MAE value that is 86.4% lower than the MAE value of the Ridge model, and the MSE value is 92.9% lower than the Ridge model. The R^2 score of the KRR model has increased massively from the Ridge Regression model, jumping from a score of 0.616 to a score of 0.974. The difference in the Ridge Regression plot and the KRR plot for both the position and orientation error distributions show this improvement, moving from a plot that has a median error of around 0.04 for both the position and orientation, to a plot with a median

position and orientation error of near 0. This could suggest that the addition of Kernel functions can massively change the way a linear model works for the better, allowing the model to learn any non-linear relationships found between the input and output data and predict results accurately.

The worst performing models appear to be the linear models, that is the Linear Regression model, the SVR model, the BLR model, the Lasso Regression model, and the Ridge Regression model. One can argue for why any one of these five models are the worst performing models; they all have similarly high MAE and MSE values, the lowest R^2 scores, and a very large spread of position and orientation errors when compared to the other models. This performance can be explained by the fact that these models cannot represent the non-linear and complex relationship between the joint angles and the desired end-effector position. One would also expect that the BLR model would perform better than the standard Linear Regression algorithm, but again it doesn't seem to be the case. The fact is that these models are linear models and cannot handle the complex relationships found within the dataset. The one thing they do have is that they are some of the fastest models to be trained and tested, taking less than a tenth of a second to be trained and tested. However, the kNN and Decision Tree models take a similar amount of time and provides significantly better results, meaning that this positive aspect does not amount to much.

4.3 Conclusions

Solving the Inverse Kinematics problem has been discussed in much detail since the invention of the industrial robot. Many solutions have been devised and come forth to try and solve this problem, with closed-form solutions using spatial geometry and systems of non-linear equations, and numerical solutions using the Jacobian matrix in different ways to find the minimum change required from the initial joint angles to the desired joint angles. With the rise of Machine Learning usage in everyday life, it makes sense to try and use these algorithms to solve this problem, and see which models are able to capture the essence of the problem and solve them with great results, and which models might miss the mark.

Therefore, in an overall case if one is looking for accurate, precise results whilst being quick to train and solve, the kNN model is the best pick. Not only does it have one of the highest R^2 scores, and some of the lowest MAE and MSE values, it is also extremely fast to train and test whilst also being precise, with a low variation in position and orientation errors. The Random Forest algorithm is also an excellent pick, despite taking a few seconds longer than the kNN model, it performs just as well.

The only setback is that these models can be very sensitive to outliers. To avoid this, the Decision Tree model can be used, as it has a high R^2 score and low MAE and MSE values. The position error and orientation error distributions also show that it is consistently close to the true values, having a very low median error and interquartile range of errors.

If one is not considered about timings, however, then the GPR model is the best choice. It has the best results out of all the models on this list, only being the most computationally expensive algorithm to run.

The models to avoid would be the linear models discussed in the section above, as these models are unable to capture the non-linear relationships found between the desired end-effector position and the joint angles.

4.4 Ideas for future work

To take this project further, looking at non-supervised algorithms, like the k-Means algorithm and PCA, and comparing their performance against the performance of the supervised models looked at in this paper could enlighten on the performance difference between supervised and non-supervised algorithms. The hyperparameter tuning would also be taken a step further. The computer used to run this project could not handle the many different parameter choices when running the hyperparameter tuning, due to resource usage and timings. Being able to integrate more choices for each parameter for all the models would ensure that the true optimal choice for each model can be found. Finally, further validation such as the method mentioned in Chapter 3.3.3 would ensure that the models are working as expected, and would be useful when including more models to the project.

Model	Best Parameters
kNN	No. of Neighbours: 2; p: 1; weight: distance
Linear Regression	Fit Intercept: False; Positive: False
Neural Network	Layer sizes: (100,); Activation: Relu; Solver: Adam; Alpha: 0.0001
Decision Trees	Max Depth: None; Min Samples Split: 5; Min Samples Leaf: 1; Max Features: None; Alpha: 0.0
SVR	C Value: 10; Epsilon Value: 0.5
Random Forest	No. of Estimators: 200; Max Depth: 20; Min Samples Split: 2; Min Samples Leaf: 1
Gradient Boosting	No. of Estimators: 70; Max Depth: None; Learning Rate: 0.1; Min Samples Split: 10; Min Samples Leaf: 5; Subsamples: 1.0; Max Features: sqrt; Loss Function: Squared Error
GPR	RationalQuadratic Kernel(Alpha=1, Length Scale=1)
BLR	Alpha 1: 8.975×10^{-6}; Alpha 2: 4.294×10^{-6}; Lambda 1: 0.0004; Lambda 2: 3.616×10^{-5}
Lasso Regression	Alpha: 0.01
Ridge Regression	Alpha: 1
KRR	Alpha: 0.01; Gamma: 10; Kernel: rbf

Table 4.1: Best Parameters for Each Model

Models	MAE (radians)	MSE (radians^2)	R^2
kNN	0.007	0.003	0.985
Linear Regression	0.162	0.085	0.616
Neural Networks	0.023	0.003	0.980
Decision Trees	0.017	0.009	0.957
SVR	0.163	0.084	0.613
Random Forest	0.016	0.005	0.978
Gradient Boosting	0.020	0.005	0.978
GPR	0.009	0.002	0.988
BLR	0.162	0.085	0.616
Lasso Regression	0.162	0.086	0.615
Ridge Regression	0.162	0.085	0.616
KRR	0.022	0.006	0.974

Table 4.2: Comparison of MAE, MSE, and R^2 Scores

Models	Training Time (s)	Testing Time (s)	Total Time (s)
kNN	0.006	0.005	0.011
Linear Regression	0.004	0.001	0.005
Neural Networks	275.126	0.024	274.150
Decision Trees	0.037	0.001	0.038
SVR	0.272	0.002	0.274
Random Forest	3.104	0.043	3.147
Gradient Boosting	110.270	0.224	110.494
GPR	1303.353	0.857	1304.210
BLR	0.032	0.004	0.036
Lasso Regression	0.010	0.001	0.011
Ridge Regression	0.003	0.001	0.004
KRR	6.546	0.152	6.698

Table 4.3: Comparison of Training and Testing Times

References

[1] A. Aristidou, J. Lasenby, Y. Chrysanthou, and A. Shamir. Inverse Kinematics Techniques in Computer Graphics: A Survey. *Computer Graphics Forum*, 2018.

[2] G. Beckwith. *Readings in Technology and Civilization*. Pearson Custom Publishing, 1997.

[3] B. Borel. A Ping-Pong-Playing Terminator. Available from: https://web.archive.org/web/20210122220024/https://www.popsci.com/technology/article/2010-02/ping-pong-playing-terminator/, 2010. Accessed: 11/02/2025.

[4] L. Breiman. Random Forests. *Machine Learning*, 2001.

[5] S. Buss. Introduction to Inverse Kinematics with Jacobian Transpose, Pseudoinverse and Damped Least Squares Methods. *IEEE Transactions in Robotics and Automation*, 2004.

[6] J. H. Challis. Quaternions as a Solution to Determining the Angular Kinematics of Human Movement. *BMC Biomedical Engineering*, 2020.

[7] Computer History Museum. Victor Scheinman's Stanford Arm. Available from: https://www.computerhistory.org/timeline/1969/#169ebbe2ad45559efbc6eb3572084edd. Accessed: 16/03/2025.

[8] J. J. Craig. Introduction to Robotics: Mechanics and Control. *Pearson*, 2005.

[9] S. Cubero. Industrial Robotics: Theory, Modelling and Control. *InTechOpen*, 2006.

[10] P. Cunningham and S. J. Delany. k-Nearest Neighbour Classifiers - A Tutorial. *ACM Comput. Surv.*, 2021.

[11] T. G. Dietterich. Ensemble Methods in Machine Learning. In *Proceedings of the First International Workshop on Multiple Classifier Systems*. Springer-Verlag, 2000.

[12] J. Funda, R. H. Taylor, and R. P. Paul. On Homogeneous Transforms, Quaternions, and Computational Efficiency. *IEEE Transactions on Robotics and Automation*, 1990.

[13] W. R. Hamilton. On Quaternions, Or On A New System Of Imaginaries In Algebra. *Philosophical Magazine*, 1844-1850.

[14] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. Springer, 2009.

[15] C.-K. Ho, L.-W. Chan, C.-T. King, and T.-y. Yen. A Deep Learning Approach to Navigating the Joint Solution Space of Redundant Inverse Kinematics and Its Applications to Numerical IK Computations. *IEEE Access*, 2023.

[16] Honda. ASIMO Featuring Intelligence Technology. Available from: https://web.archive.org/web/20070930181716/http://world.honda.com/ASIMO/technology/intelligence.html, 2002. Accessed: 21/03/2025.

[17] G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning*. Springer New York, NY, 2013.

[18] P. M. Kebria, S. Al-wais, H. Abdi, and S. Nahavandi. Kinematic and Dynamic Modelling of UR5 Manipulator. In *2016 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2016.

[19] J.-H. Kim and V. R. Kumar. Kinematics of Robot Manipulators Via Line Transformations. *Journal of Robotic Systems*, 1990.

[20] A. P. Kotelnikov. Screw Calculus and Some of its Applications to Geometry and Mechanics. *Annals of the Imperial University of Kazan*, 1895.

[21] A. Krishnan. *Killer Robots: Legality and Ethicality of Autonomous Weapons*. Ashgate Publishing, 2009.

[22] S. Kucuk and Z. Bingul. Robot Kinematics: Forward and Inverse Kinematics. In *Industrial Robotics*. IntechOpen, 2006.

[23] B. Mahesh. Machine Learning Algorithms. *International Journal of Science and Research (IJSR)*, 2018.

[24] D. Mienye and N. Jere. A Survey of Decision Trees: Concepts, Algorithms, and Applications. *IEEE Access*, 2024.

[25] C. Mishra and D. Gupta. Deep Machine Learning and Neural Networks: An Overview. *IAES International Journal of Artificial Intelligence (IJ-AI)*, 2017.

[26] M. E. Moran. The da Vinci Robot. *J Endourol.*, 2006.

[27] K. Murphy. *Machine Learning: A Probabilistic Perspective*. The MIT Press, 2013.

[28] National Inventors Hall of Fame. George Devol. Available from: https://www.invent.org/inductees/george-devol, 2011. Accessed: 16/03/2025.

[29] L. Nocks. *The Robot: The Life Story of a Technology*. Bloomsbury Academic, 2007.

[30] Oxford Dictionary. Robotics definition.

[31] J. A. Perez, F. Deligianni, D. Ravi, and G.-Z. Yang. Artificial Intelligence and Robotics. *arXiv preprint arXiv:1803.10813*, 2018.

[32] C. Rasmussen and C. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 2006.

[33] M. R. Reese. The Steam-Powered Pigeon of Archytas – The Flying Machine of Antiquity. Available from: https://www.ancient-origins.net/history-famous-people/steam-powered-pigeon-002179, 2021. Accessed: 12/03/2025.

[34] A. Samuel. Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development*, 1959.

[35] M. Schick. KR FAMULUS - Industrial Robot Ahead of its Time. Available from: https://www.kuka.com/en-gb/company/iimagazine/2023/12/homeofrobotik-robotikgeschichte, 2023. Accessed: 16/03/2025.

[36] P. C. Sen, M. Hajra, and M. Ghosh. Supervised Classification Algorithms in Machine Learning: A Survey and Review. In *Emerging Technology in Modelling and Graphics*. Springer Singapore, 2020.

[37] A. Sherbiny, M. Elhosseini, and A. Haikal. A Comparative Study of Soft Computing Methods To Solve Inverse Kinematics Problem. *Ain Shams Engineering Journal*, 2017.

[38] F. Siddiqui. Tesla says it is building a 'friendly' robot that will perform menial tasks, won't fight back. Available from: https://www.washingtonpost.com/technology/2021/08/19/tesla-ai-day-robot/, 2021. Accessed: 16/03/2025.

[39] A. Smola and B. Schölkopf. A Tutorial on Support Vector Regression. *Statistics and computing*, 2004.

[40] F. Stulp and O. Sigaud. Many Regression Algorithms, One Unified Model: A Review. *Neural Networks*, 2015.

[41] University of Bristol. Grey Walter and his tortoises. Available from: https://www.bristol.ac.uk/news/2008/212017945378.html, 2008. Accessed: 16/03/2025.

[42] A. Williams. *History of Digital Games*. CRC Press, 2017.

Appendix A

Self-appraisal

A.1 Critical self-evaluation

I was interested in basing my dissertation project on something within the field of robotics and machine learning, but wasn't sure which topic would allow me to do both. When I saw the inverse kinematics topic listed on the topic list under my supervisor's name, I was interested in learning more. After some research, it became clear to me that this is a topic that which would allow me to use machine learning within the robotics field.

At first, I generated my own dataset to use for training and testing the models. This was good because it allowed me to see what kind of values I should be looking out for, but because I generated it every time the project was ran, I would get different results, which wasn't helpful as it wouldn't be reproducible. Thankfully, I was able to get a dataset of real trajectories that I could use, and would develop reproducible results.

When implementing the models, I realised that each of the models have a lot of different parameters that could be changed to provide a better prediction. I then found the `GridSearchCV` function which I then used, alongside a Python dictionary which stored the different parameters and their possible choices. Many of these parameters were integer values, but I couldn't use a list to loop through each value from 1 to n as it would have taken too long, and my computer wouldn't have the resources to run for that long. For parameters like k in the kNN model, this works, but for the learning rate in the Gradient Boosting model, it would have taken too long. So I chose to sample a select few values for these parameters.

Despite this change, the tuning was taking extremely long, especially for models such as the Neural Networks model, the Gaussian Process Regression Model, and the Gradient Boosting model. This is because the `GridSearchCV` function was checking every possible combination of these parameters, and these models had a lot of different parameters, leading to many different combinations. To reduce the time taken, I used the `RandomizedSearchCV` function, which looks at a random subset of parameters. This helped reduce the time taken to find the best parameters for these models.

The next obstacle I had was calculating the position and orientation error of the models. At first, the violin plots for each of the models were exactly the same, meaning that the way the errors were being calculated was incorrect. After taking some time to go through and correct the error, I was able to solve it so the violin plots were not the same for all the models.

At the end of this project, I was able to meet my target of looking at different machine learning algorithms and analysing how they solved the inverse kinematics problem. Whilst I may not

have been able to get the optimal version of each model, I was still able to find some insight into which model did better in certain areas and why that might have been.

A.2 Personal reflection and lessons learned

I had never really worked with robotics before, but I was interested in knowing how they worked. Undertaking this project taught me that inverse kinematics is a key aspect of this field, and is something that is still being looked into to this day. I would not have known about this topic, nor have contributed to it, if I wasn't drawn to the field of Robotics.

This project also helped me learn more about machine learning. Having learnt the theory behind it, as well as some different models (some included within this project), I was able to take that learning and apply it to a real problem. I was able to see different models that I would not have normally come across, and learn more about how to measure the success of a model, and why some may be more successful than others.

Overall, I was able to learn about a very interesting field of robotics that is still having discussions on how to solve, and learn more about machine learning and how to evaluate models. Being able to intersect the two fields has shown me just how interconnected the field of Computer Science can be, and has given me newfound knowledge that I will pursue further, whether it be in an academic environment or a working environment.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

This project uses the PyBullet library, a third-party library that is used for physics and robotics simulation. When using this library, one should ensure they meet the Terms of Use of this library, especially if the product is to be commercialised or distributed. This project meets these conditions.

A.3.2 Social issues

This project is primarily technical research into using pre-existing algorithms and applying them to a new situation, meaning the social impact is minimal, However, one could consider access to hardware needed to run the project or use the findings from this project, which may not be accessible to everyone.

A.3.3 Ethical issues

This project does not include any human beings, animals, or other such living things, nor is any personal information used, meaning that the project is of low ethical risk.

A.3.4 Professional issues

This project is an academic-based project, meaning that the professional standards met are those of academic standards. Ensuring proper citations for academic integrity, use of best software development practices, and reproducibility are crucial.

Appendix B

External Material

The dataset was prepared by Shaokang Wu, whom I was able to get in contact with via my supervisor, Dr Yanlong Huang.

The robotic simulator used is by Deepak Raina, and found in the following GitHub repository: https://github.com/deepakraina99/pybullet-kinematics-dynamics-control/

The URDF file used for the simulator is by the Stanfor Autonomous Systems Lab, and is found within the following GitHub repository: https://github.com/StanfordASL/PandaRobot.jl

Appendix C

Additional Information

This project was run on a PC with the following specifications:

- Processor: AMD Ryzen 5 5600X 6-Core Processor @ 3.70 GHz
- Installed RAM: 16.0 GB
- Graphics Card: NVIDIA GeForce RTX 4060
- System type: 64-bit operating system, x64-based processor
- Operating System: Windows 11 Pro V24H2